

COMP102 Lecture 10: Sorting Algorithms

Merge sort, quicksort, complexity analysis

Jalal Maaouni

June 7, 2026



University
Mohammed VI
Polytechnic

THE UM6P
VANGUARD
CENTER

#Empowering Minds.

Plan

1 Recap and target

2 Merge sort

3 The Master Theorem

4 Merge sort Complexity

5 Quicksort

6 Quicksort complexity

7 Comparison and practice

RECAP AND TARGET

What we already know

From first semester

We have already seen basic sorting and the rough growth classes: linear, quadratic, and $n \log n$.

What we already know

From first semester

We have already seen basic sorting and the rough growth classes: linear, quadratic, and $n \log n$.

Today

We focus on the two efficient divide-and-conquer sorts:

What we already know

From first semester

We have already seen basic sorting and the rough growth classes: linear, quadratic, and $n \log n$.

Today

We focus on the two efficient divide-and-conquer sorts:

- ▶ merge sort;

What we already know

From first semester

We have already seen basic sorting and the rough growth classes: linear, quadratic, and $n \log n$.

Today

We focus on the two efficient divide-and-conquer sorts:

- ▶ merge sort;
- ▶ quicksort;

What we already know

From first semester

We have already seen basic sorting and the rough growth classes: linear, quadratic, and $n \log n$.

Today

We focus on the two efficient divide-and-conquer sorts:

- ▶ merge sort;
- ▶ quicksort;
- ▶ their implementation details;

What we already know

From first semester

We have already seen basic sorting and the rough growth classes: linear, quadratic, and $n \log n$.

Today

We focus on the two efficient divide-and-conquer sorts:

- ▶ merge sort;
- ▶ quicksort;
- ▶ their implementation details;
- ▶ their best, average, and worst-case complexity.

Quick recap: selection and insertion sort

Selection sort

Repeatedly select the minimum from the unsorted part and swap it into the next position.(Animation)

```
1 for i in range(n):
2     min_index = i
3     for j in range(i + 1, n):
4         if a[j] < a[min_index]:
5             min_index = j
6     swap(a[i], a[min_index])
```

Insertion sort

Keep a sorted prefix and insert each new element into that prefix.(Animation)

```
1 for i in range(1, n):
2     key = a[i]
3     j = i - 1
4     while j >= 0 and a[j] > key:
5         a[j + 1] = a[j]
6         j -= 1
7     a[j + 1] = key
```

Why quadratic sorting is not enough

n	n^2	$n \log_2 n$ approximately
10	100	33
100	10 000	664
1 000	1 000 000	9 966
1 000 000	10^{12}	19 931 569

Why quadratic sorting is not enough

n	n^2	$n \log_2 n$ approximately
10	100	33
100	10 000	664
1 000	1 000 000	9 966
1 000 000	10^{12}	19 931 569

For large input, $O(n^2)$ is not a small slowdown. It is a different scale of computation.

Divide and conquer for sorting

Definition 1.1 Divide and conquer

Divide the problem into smaller problems, solve the smaller problems, then combine the result.

Divide and conquer for sorting

Definition 1.2 Divide and conquer

Divide the problem into smaller problems, solve the smaller problems, then combine the result.

Generic shape

Divide and conquer for sorting

Definition 1.3 Divide and conquer

Divide the problem into smaller problems, solve the smaller problems, then combine the result.

Generic shape

▶ divide;

Divide and conquer for sorting

Definition 1.4 Divide and conquer

Divide the problem into smaller problems, solve the smaller problems, then combine the result.

Generic shape

- ▶ divide;
- ▶ recursively solve;

Divide and conquer for sorting

Definition 1.5 Divide and conquer

Divide the problem into smaller problems, solve the smaller problems, then combine the result.

Generic shape

- ▶ divide;
- ▶ recursively solve;
- ▶ combine.

Divide and conquer for sorting

Definition 1.6 Divide and conquer

Divide the problem into smaller problems, solve the smaller problems, then combine the result.

Generic shape

- ▶ divide;
- ▶ recursively solve;
- ▶ combine.

Merge sort and quicksort are both divide-and-conquer algorithms, but the expensive linear step happens in different places.

MERGE SORT

Merge sort: idea

Definition 2.1 Merge sort

Merge sort splits the list into two halves, recursively sorts both halves, then merges the two sorted halves.

Merge sort: idea

Definition 2.2 Merge sort

Merge sort splits the list into two halves, recursively sorts both halves, then merges the two sorted halves.

Algorithm

Merge sort: idea

Definition 2.3 Merge sort

Merge sort splits the list into two halves, recursively sorts both halves, then merges the two sorted halves.

Algorithm

- ▶ split the list in half;

Merge sort: idea

Definition 2.4 Merge sort

Merge sort splits the list into two halves, recursively sorts both halves, then merges the two sorted halves.

Algorithm

- ▶ split the list in half;
- ▶ sort the left half;

Merge sort: idea

Definition 2.5 Merge sort

Merge sort splits the list into two halves, recursively sorts both halves, then merges the two sorted halves.

Algorithm

- ▶ split the list in half;
- ▶ sort the left half;
- ▶ sort the right half;

Merge sort: idea

Definition 2.6 Merge sort

Merge sort splits the list into two halves, recursively sorts both halves, then merges the two sorted halves.

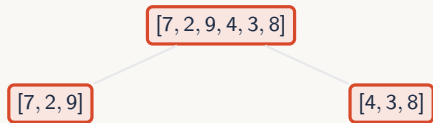
Algorithm

- ▶ split the list in half;
- ▶ sort the left half;
- ▶ sort the right half;
- ▶ merge the two sorted halves.

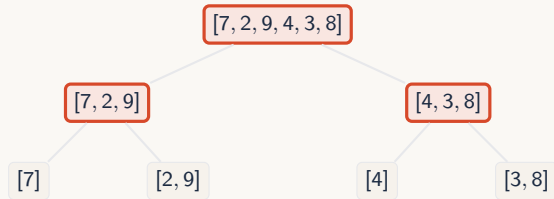
Merge sort: splitting

[7, 2, 9, 4, 3, 8]

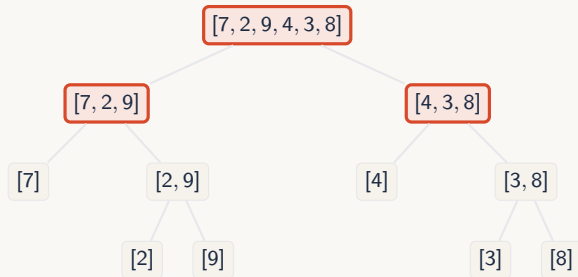
Merge sort: splitting



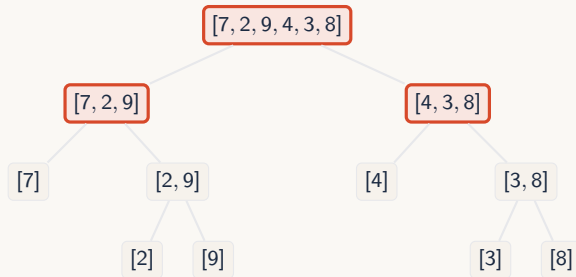
Merge sort: splitting



Merge sort: splitting



Merge sort: splitting



A list of size one is already sorted. That is the base case of merge sort.

Merge sort: merging

Merge two sorted lists

[2, 7, 9] and [3, 4, 8]

left

2

7

9

right

3

4

8

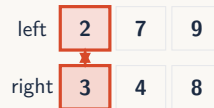
result:

Merge sort: merging

Merge two sorted lists

[2, 7, 9] and [3, 4, 8]

- ▶ compare the first unused elements;



result:

Merge sort: merging

Merge two sorted lists

[2, 7, 9] and [3, 4, 8]

- ▶ compare the first unused elements;
- ▶ copy the smaller one;



Merge sort: merging

Merge two sorted lists

[2, 7, 9] and [3, 4, 8]

- ▶ compare the first unused elements;
- ▶ copy the smaller one;
- ▶ move the pointer in that list;

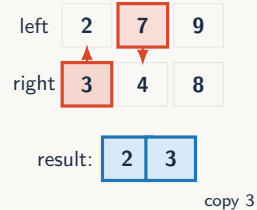


Merge sort: merging

Merge two sorted lists

[2, 7, 9] and [3, 4, 8]

- ▶ compare the first unused elements;
- ▶ copy the smaller one;
- ▶ move the pointer in that list;
- ▶ continue comparing unused elements;

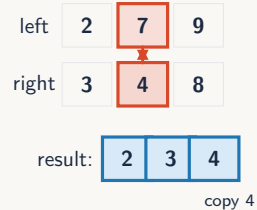


Merge sort: merging

Merge two sorted lists

[2, 7, 9] and [3, 4, 8]

- ▶ compare the first unused elements;
- ▶ copy the smaller one;
- ▶ move the pointer in that list;
- ▶ continue comparing unused elements;

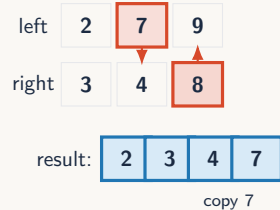


Merge sort: merging

Merge two sorted lists

[2, 7, 9] and [3, 4, 8]

- ▶ compare the first unused elements;
- ▶ copy the smaller one;
- ▶ move the pointer in that list;
- ▶ continue comparing unused elements;

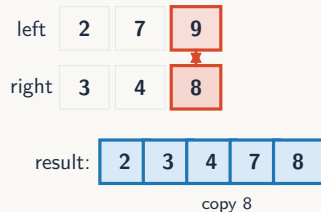


Merge sort: merging

Merge two sorted lists

[2, 7, 9] and [3, 4, 8]

- ▶ compare the first unused elements;
- ▶ copy the smaller one;
- ▶ move the pointer in that list;
- ▶ continue comparing unused elements;
- ▶ append the remaining elements.

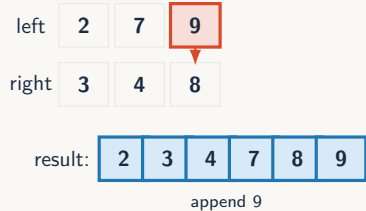


Merge sort: merging

Merge two sorted lists

[2, 7, 9] and [3, 4, 8]

- ▶ compare the first unused elements;
- ▶ copy the smaller one;
- ▶ move the pointer in that list;
- ▶ continue comparing unused elements;
- ▶ append the remaining elements.



Coding: merge two sorted lists

```
1 def merge(left, right):
2     result = []
3     i = 0
4     j = 0
5
6     while i < len(left) and j < len(right):
7         if left[i] <= right[j]:
8             result.append(left[i])
9             i += 1
10        else:
11            result.append(right[j])
12            j += 1
13
14    result.extend(left[i:])
15    result.extend(right[j:])
16    return result
```

Coding: merge two sorted lists

```
1 def merge(left, right):
2     result = []
3     i = 0
4     j = 0
5
6     while i < len(left) and j < len(right):
7         if left[i] <= right[j]:
8             result.append(left[i])
9             i += 1
10        else:
11            result.append(right[j])
12            j += 1
13
14    result.extend(left[i:])
15    result.extend(right[j:])
16    return result
```

The merge step is linear: every element is copied into the result exactly once.

Coding: merge sort

```
1 def merge_sort(a):  
2     if len(a) <= 1:  
3         return a  
4  
5     middle = len(a) // 2  
6     left = merge_sort(a[:middle])  
7     right = merge_sort(a[middle:])  
8  
9     return merge(left, right)
```

Coding: merge sort

```
1 def merge_sort(a):  
2     if len(a) <= 1:  
3         return a  
4  
5     middle = len(a) // 2  
6     left = merge_sort(a[:middle])  
7     right = merge_sort(a[middle:])  
8  
9     return merge(left, right)
```

Important implementation note

This version is written for clarity. Slicing creates new lists, so it is not the lowest-level possible implementation.

Coding: merge sort

```
1 def merge_sort(a):  
2     if len(a) <= 1:  
3         return a  
4  
5     middle = len(a) // 2  
6     left = merge_sort(a[:middle])  
7     right = merge_sort(a[middle:])  
8  
9     return merge(left, right)
```

Important implementation note

This version is written for clarity. Slicing creates new lists, so it is not the lowest-level possible implementation.

For this course, the key idea is the recursive structure and the linear merge step.

THE MASTER THEOREM

Master Theorem: when to use it

Standard form

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

- ▶ a : number of recursive subproblems;
- ▶ n/b : size of each subproblem;
- ▶ $f(n)$: work done outside recursion;
- ▶ applies when subproblem sizes shrink by a constant factor.

Master Theorem: when to use it

Standard form

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

- ▶ a : number of recursive subproblems;
- ▶ n/b : size of each subproblem;
- ▶ $f(n)$: work done outside recursion;
- ▶ applies when subproblem sizes shrink by a constant factor.

Warning: It does not apply to recurrences like:

$$T(n) = T(n - 1) + cn$$

Master Theorem: the comparison

For

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

compute:

$$n^{\log_b a}$$

Then compare:

$$f(n) \quad \text{with} \quad n^{\log_b a}$$

Master Theorem: the comparison

For

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

compute:

$$n^{\log_b a}$$

Then compare:

$$f(n) \quad \text{with} \quad n^{\log_b a}$$

Meaning

$n^{\log_b a}$ represents the total recursive branching cost.

Master Theorem: three cases

Case	Condition	Result
1	$f(n)$ smaller	$T(n) = \Theta(n^{\log_b a})$
2	$f(n) = \Theta(n^{\log_b a})$	$T(n) = \Theta(n^{\log_b a} \log n)$
3	$f(n)$ larger	$T(n) = \Theta(f(n))$

Master Theorem: three cases

Case	Condition	Result
1	$f(n)$ smaller	$T(n) = \Theta(n^{\log_b a})$
2	$f(n) = \Theta(n^{\log_b a})$	$T(n) = \Theta(n^{\log_b a} \log n)$
3	$f(n)$ larger	$T(n) = \Theta(f(n))$

Practical version: Compare the combine work with the recursion-tree growth.

MERGE SORT COMPLEXITY

Merge sort complexity: recurrence

One call on input size n

Merge sort complexity: recurrence

One call on input size n

- ▶ solve the left half: $T(n/2)$;

Merge sort complexity: recurrence

One call on input size n

- ▶ solve the left half: $T(n/2)$;
- ▶ solve the right half: $T(n/2)$;

Merge sort complexity: recurrence

One call on input size n

- ▶ solve the left half: $T(n/2)$;
- ▶ solve the right half: $T(n/2)$;
- ▶ merge both halves: cn .

Merge sort complexity: recurrence

One call on input size n

- ▶ solve the left half: $T(n/2)$;
- ▶ solve the right half: $T(n/2)$;
- ▶ merge both halves: cn .

$$T(n) = 2T(n/2) + cn$$

Merge sort complexity: recurrence

One call on input size n

- ▶ solve the left half: $T(n/2)$;
- ▶ solve the right half: $T(n/2)$;
- ▶ merge both halves: cn .

$$T(n) = 2T(n/2) + cn$$

The recurrence is the same in best, average, and worst case, because merge sort always splits by position.

Merge sort complexity: Master Theorem

Master Theorem form

$$T(n) = aT(n/b) + f(n)$$

Merge sort complexity: Master Theorem

Master Theorem form

$$T(n) = aT(n/b) + f(n)$$

For merge sort

$$T(n) = 2T(n/2) + cn$$

$$a = 2$$

$$b = 2$$

$$f(n) = cn = \Theta(n)$$

Merge sort complexity: Master Theorem

Master Theorem form

$$T(n) = aT(n/b) + f(n)$$

For merge sort

$$T(n) = 2T(n/2) + cn$$

$$a = 2$$

$$b = 2$$

$$f(n) = cn = \Theta(n)$$

Merge sort complexity: Master Theorem

Master Theorem form

$$T(n) = aT(n/b) + f(n)$$

For merge sort

$$T(n) = 2T(n/2) + cn$$

$$a = 2$$

$$b = 2$$

$$f(n) = cn = \Theta(n)$$

Master Theorem Case 2: $T(n) = \Theta(n \log n)$.

Merge sort: final properties

Property	Merge sort
Best case	$\Theta(n \log n)$
Average case	$\Theta(n \log n)$
Worst case	$\Theta(n \log n)$
Extra memory	$\Theta(n)$
Stable	Yes, if equal elements are merged from the left first

QUICKSORT

Animation

Definition 5.1 Quicksort

Quicksort chooses a pivot, partitions the array around it, then recursively sorts the left and right parts.

Quicksort with extra lists: only for intuition

```
1 def quicksort_visual(a):
2     if len(a) <= 1:
3         return a
4
5     pivot = a[len(a) // 2]
6     smaller = []
7     equal = []
8     larger = []
9
10    for value in a:
11        if value < pivot:
12            smaller.append(value)
13        elif value > pivot:
14            larger.append(value)
15        else:
16            equal.append(value)
17
18    return quicksort_visual(smaller) + equal + quicksort_visual(larger)
```

Quicksort: partition around a pivot

Input

[7, 2, 9, 4, 3, 8]

Choose pivot 4.



partition



Quicksort: partition around a pivot

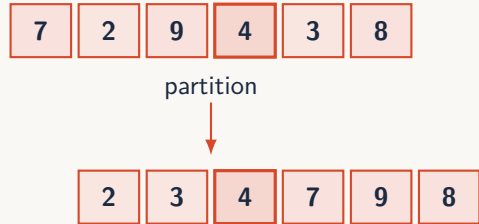
Input

[7, 2, 9, 4, 3, 8]

Choose pivot 4.

After partitioning

[2, 3] [4] [7, 9, 8]



Quicksort: partition around a pivot

Input

[7, 2, 9, 4, 3, 8]

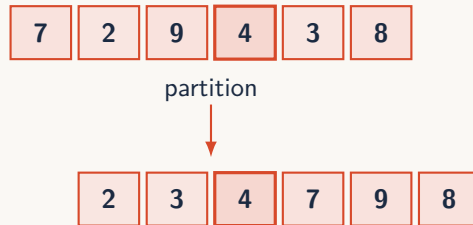
Choose pivot 4.

After partitioning

[2, 3] [4] [7, 9, 8]

Key fact

The pivot is now in its final sorted position.

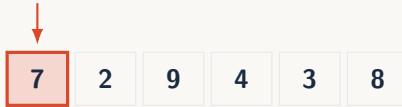


In-place partition: meaning of the indices

7	2	9	4	3	8
---	---	---	---	---	---

In-place partition: meaning of the indices

pivot



Pivot

The pivot is the first element of the current segment: $a[\text{low}]$. It stays there until the final swap.

In-place partition: meaning of the indices



Pivot

The pivot is the first element of the current segment: $a[\text{low}]$. It stays there until the final swap.

Index i

Position where the next value less than or equal to the pivot should be placed.

In-place partition: meaning of the indices



Pivot

The pivot is the first element of the current segment: $a[\text{low}]$. It stays there until the final swap.

Index i

Position where the next value less than or equal to the pivot should be placed.

Index j

Scans the current segment from left to right, starting just after the pivot.

In-place partition: meaning of the indices



Pivot

The pivot is the first element of the current segment: $a[\text{low}]$. It stays there until the final swap.

Index i

Position where the next value less than or equal to the pivot should be placed.

Index j

Scans the current segment from left to right, starting just after the pivot.

Final step

At the end, the pivot is swapped with the element just before i , so its final position is $i - 1$.

Coding: in-place partition

```
1 def partition(a, low, high):
2     pivot = a[low]
3     i = low + 1
4
5     for j in range(low + 1, high + 1):
6         if a[j] <= pivot:
7             a[i], a[j] = a[j], a[i]
8             i += 1
9
10    pivot_index = i - 1
11    a[low], a[pivot_index] = a[pivot_index], a[low]
12    return pivot_index
```

Coding: in-place partition

```
1 def partition(a, low, high):
2     pivot = a[low]
3     i = low + 1
4
5     for j in range(low + 1, high + 1):
6         if a[j] <= pivot:
7             a[i], a[j] = a[j], a[i]
8             i += 1
9
10    pivot_index = i - 1
11    a[low], a[pivot_index] = a[pivot_index], a[low]
12    return pivot_index
```

The pivot starts at the first position. After one scan, all values less than or equal to the pivot are placed before i , then the pivot is moved to $i - 1$.

Coding: in-place partition

```
1 def partition(a, low, high):
2     pivot = a[low]
3     i = low + 1
4
5     for j in range(low + 1, high + 1):
6         if a[j] <= pivot:
7             a[i], a[j] = a[j], a[i]
8             i += 1
9
10    pivot_index = i - 1
11    a[low], a[pivot_index] = a[pivot_index], a[low]
12    return pivot_index
```

The pivot starts at the first position. After one scan, all values less than or equal to the pivot are placed before i , then the pivot is moved to $i - 1$.

Partition scans the segment once. Therefore, partitioning a segment of size n costs $\Theta(n)$.

Coding: in-place quicksort

```
1 def quicksort(a, low=0, high=None):
2     if high is None:
3         high = len(a) - 1
4
5     if low >= high:
6         return
7
8     pivot_index = partition(a, low, high)
9
10    quicksort(a, low, pivot_index - 1)
11    quicksort(a, pivot_index + 1, high)
```

Coding: in-place quicksort

```
1 def quicksort(a, low=0, high=None):
2     if high is None:
3         high = len(a) - 1
4
5     if low >= high:
6         return
7
8     pivot_index = partition(a, low, high)
9
10    quicksort(a, low, pivot_index - 1)
11    quicksort(a, pivot_index + 1, high)
```

Unlike merge sort, quicksort does not merge after recursion. The pivot is already fixed after partitioning.

Merge sort vs quicksort structure

Question	Merge sort	Quicksort
How does it divide?	By position	By pivot value
When is linear work done?	After recursion	Before recursion
Main operation	Merge	Partition
Is there a final merge?	Yes	No

Merge sort vs quicksort structure

Question	Merge sort	Quicksort
How does it divide?	By position	By pivot value
When is linear work done?	After recursion	Before recursion
Main operation	Merge	Partition
Is there a final merge?	Yes	No

Merge sort organizes the output during merging. Quicksort organizes the array during partitioning.

QUICKSORT COMPLEXITY

Quicksort complexity starts with partition

Cost of one partition

Partition scans the current segment once.

$$\Theta(n)$$



good pivot

Quicksort complexity starts with partition

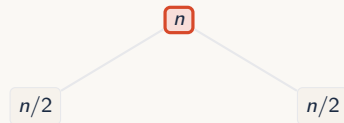
Cost of one partition

Partition scans the current segment once.

$$\Theta(n)$$

What changes between cases?

The partition cost is always linear for one call. The difference is the size of the recursive subproblems.



good pivot

Quicksort complexity starts with partition

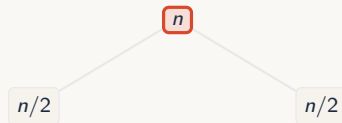
Cost of one partition

Partition scans the current segment once.

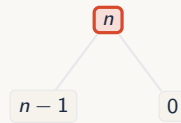
$$\Theta(n)$$

What changes between cases?

The partition cost is always linear for one call. The difference is the size of the recursive subproblems.



good pivot



bad pivot

Best case quicksort: Master Theorem

Best case assumption

The pivot always splits the array into two equal halves.

$$T(n) = 2T(n/2) + cn$$

Best case quicksort: Master Theorem

Best case assumption

The pivot always splits the array into two equal halves.

$$T(n) = 2T(n/2) + cn$$

Match the Master Theorem

$$\begin{aligned}T(n) &= aT(n/b) + f(n) \\T(n) &= 2T(n/2) + cn \\a &= 2 \\b &= 2 \\f(n) &= cn = \Theta(n)\end{aligned}$$

Best case quicksort: Master Theorem

Best case assumption

The pivot always splits the array into two equal halves.

$$T(n) = 2T(n/2) + cn$$

Match the Master Theorem

$$\begin{aligned}T(n) &= aT(n/b) + f(n) \\T(n) &= 2T(n/2) + cn \\a &= 2 \\b &= 2 \\f(n) &= cn = \Theta(n)\end{aligned}$$

Best case quicksort: Master Theorem

Best case assumption

The pivot always splits the array into two equal halves.

$$T(n) = 2T(n/2) + cn$$

Match the Master Theorem

$$\begin{aligned}T(n) &= aT(n/b) + f(n) \\T(n) &= 2T(n/2) + cn \\a &= 2 \\b &= 2 \\f(n) &= cn = \Theta(n)\end{aligned}$$

Master Theorem Case 2: best-case quicksort is $\Theta(n \log n)$.

Worst case quicksort

Worst case assumption

The pivot is always the smallest or largest element.

$$T(n) = T(n - 1) + cn$$

Worst case quicksort

Worst case assumption

The pivot is always the smallest or largest element.

$$T(n) = T(n - 1) + cn$$

Why Master Theorem does not apply

Master Theorem expects subproblems of size n/b :

$$T(n) = aT(n/b) + f(n)$$

Here the subproblem has size $n - 1$, not n/b .

Worst case quicksort

Start

$$T(n) = T(n - 1) + cn$$

Worst case quicksort

Start

$$T(n) = T(n-1) + cn$$

Expand repeatedly

$$\begin{aligned}T(n) &= T(n-1) + cn \\&= T(n-2) + c(n-1) + cn \\&= T(n-3) + c(n-2) + c(n-1) + cn \\&\dots \\&= c(1 + 2 + 3 + \dots + n)\end{aligned}$$

Worst case quicksort

Start

$$T(n) = T(n-1) + cn$$

Expand repeatedly

$$\begin{aligned}T(n) &= T(n-1) + cn \\&= T(n-2) + c(n-1) + cn \\&= T(n-3) + c(n-2) + c(n-1) + cn \\&\dots \\&= c(1 + 2 + 3 + \dots + n)\end{aligned}$$

Use the arithmetic sum

$$1 + 2 + 3 + \dots + n = \frac{n(n+1)}{2}$$

Worst case quicksort

Start

$$T(n) = T(n-1) + cn$$

Expand repeatedly

$$\begin{aligned}T(n) &= T(n-1) + cn \\&= T(n-2) + c(n-1) + cn \\&= T(n-3) + c(n-2) + c(n-1) + cn \\&\dots \\&= c(1 + 2 + 3 + \dots + n)\end{aligned}$$

Use the arithmetic sum

$$1 + 2 + 3 + \dots + n = \frac{n(n+1)}{2}$$

Worst-case quicksort is $\Theta(n^2)$.

Average case quicksort

Strict average-case recurrence

If all pivot ranks are equally likely:

$$T(n) = cn + \frac{1}{n} \sum_{k=0}^{n-1} (T(k) + T(n-1-k))$$

Average case quicksort

Strict average-case recurrence

If all pivot ranks are equally likely:

$$T(n) = cn + \frac{1}{n} \sum_{k=0}^{n-1} (T(k) + T(n-1-k))$$

The exact average-case proof is not a direct Master Theorem application, but it also gives $\Theta(n \log n)$.

Average-case quicksort: recurrence

$$T(n) = cn + \frac{1}{n} \sum_{k=0}^{n-1} (T(k) + T(n-1-k))$$

$$T(n) = cn + \frac{2}{n} \sum_{k=0}^{n-1} T(k)$$

$$nT(n) = cn^2 + 2 \sum_{k=0}^{n-1} T(k) \tag{1}$$

Average-case quicksort: shift to $n - 1$

$$(n - 1)T(n - 1) = c(n - 1)^2 + 2 \sum_{k=0}^{n-2} T(k) \quad (2)$$

$$(1) - (2)$$

$$nT(n) - (n - 1)T(n - 1) = c(n^2 - (n - 1)^2) + 2T(n - 1)$$

Average-case quicksort: simplify

$$n^2 - (n - 1)^2 = 2n - 1$$

$$nT(n) - (n - 1)T(n - 1) = c(2n - 1) + 2T(n - 1)$$

$$nT(n) = (n + 1)T(n - 1) + c(2n - 1)$$

Average-case quicksort: normalize

$$\frac{T(n)}{n+1} = \frac{T(n-1)}{n} + \frac{c(2n-1)}{n(n+1)}$$

$$\frac{c(2n-1)}{n(n+1)} = \Theta\left(\frac{1}{n}\right)$$

$$\frac{T(n)}{n+1} = \frac{T(n-1)}{n} + \Theta\left(\frac{1}{n}\right)$$

Average-case quicksort: telescope

$$\frac{T(n)}{n+1} = \Theta\left(1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n}\right)$$

$$1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n} = \Theta(\log n)$$

$$T(n) = \Theta(n \log n)$$

Quicksort: avoiding the worst case in practice

Problem

Quicksort becomes slow when the pivot repeatedly gives very unbalanced partitions.

$$T(n) = T(n - 1) + cn = \Theta(n^2)$$

Quicksort: avoiding the worst case in practice

Problem

Quicksort becomes slow when the pivot repeatedly gives very unbalanced partitions.

$$T(n) = T(n-1) + cn = \Theta(n^2)$$

Practical trick: shuffle first

Before running quicksort, randomly shuffle the array.

$$\text{shuffle}(a) \implies \text{quicksort}(a)$$

Quicksort: avoiding the worst case in practice

Problem

Quicksort becomes slow when the pivot repeatedly gives very unbalanced partitions.

$$T(n) = T(n - 1) + cn = \Theta(n^2)$$

Practical trick: shuffle first

Before running quicksort, randomly shuffle the array.

$$\text{shuffle}(a) \implies \text{quicksort}(a)$$

After shuffling, the input order is random, so a fixed pivot rule such as “choose the first element” behaves like choosing a random pivot.

Quicksort: avoiding the worst case in practice

Problem

Quicksort becomes slow when the pivot repeatedly gives very unbalanced partitions.

$$T(n) = T(n-1) + cn = \Theta(n^2)$$

Practical trick: shuffle first

Before running quicksort, randomly shuffle the array.

$$\text{shuffle}(a) \implies \text{quicksort}(a)$$

After shuffling, the input order is random, so a fixed pivot rule such as “choose the first element” behaves like choosing a random pivot.

- makes adversarial or already sorted inputs unlikely to trigger the worst case;

Quicksort: avoiding the worst case in practice

Problem

Quicksort becomes slow when the pivot repeatedly gives very unbalanced partitions.

$$T(n) = T(n-1) + cn = \Theta(n^2)$$

Practical trick: shuffle first

Before running quicksort, randomly shuffle the array.

$$\text{shuffle}(a) \implies \text{quicksort}(a)$$

After shuffling, the input order is random, so a fixed pivot rule such as “choose the first element” behaves like choosing a random pivot.

- ▶ makes adversarial or already sorted inputs unlikely to trigger the worst case;
- ▶ pushes behavior closer to the average case;

Quicksort: properties

Property	In-place quicksort
Best case	$\Theta(n \log n)$
Average case	$\Theta(n \log n)$
Worst case	$\Theta(n^2)$
Extra memory, balanced recursion	$\Theta(\log n)$ stack
Extra memory, worst recursion	$\Theta(n)$ stack
Stable	Usually no

Quicksort: properties

Property	In-place quicksort
Best case	$\Theta(n \log n)$
Average case	$\Theta(n \log n)$
Worst case	$\Theta(n^2)$
Extra memory, balanced recursion	$\Theta(\log n)$ stack
Extra memory, worst recursion	$\Theta(n)$ stack
Stable	Usually no

Quicksort is usually fast, but pivot choice controls whether the recursion tree is balanced or degenerate.

COMPARISON AND PRACTICE

Merge sort vs in-place quicksort

Feature	Merge sort	In-place quicksort
Best case	$\Theta(n \log n)$	$\Theta(n \log n)$
Average case	$\Theta(n \log n)$	$\Theta(n \log n)$
Worst case	$\Theta(n \log n)$	$\Theta(n^2)$
Main operation	merge	partition
Extra memory	$\Theta(n)$	$\Theta(\log n)$ average stack
Stable	yes, if careful	usually no
Risk	memory cost	bad pivots

Merge sort vs in-place quicksort

Feature	Merge sort	In-place quicksort
Best case	$\Theta(n \log n)$	$\Theta(n \log n)$
Average case	$\Theta(n \log n)$	$\Theta(n \log n)$
Worst case	$\Theta(n \log n)$	$\Theta(n^2)$
Main operation	merge	partition
Extra memory	$\Theta(n)$	$\Theta(\log n)$ average stack
Stable	yes, if careful	usually no
Risk	memory cost	bad pivots

Merge sort is more predictable. Quicksort is more sensitive to pivot quality, but its in-place version is memory-efficient and often fast in practice.

What to remember

- ▶ Selection and insertion sort are useful baselines, but their average and worst cases are quadratic.

What to remember

- ▶ Selection and insertion sort are useful baselines, but their average and worst cases are quadratic.
- ▶ Merge sort splits by position and merges sorted halves.

What to remember

- ▶ Selection and insertion sort are useful baselines, but their average and worst cases are quadratic.
- ▶ Merge sort splits by position and merges sorted halves.
- ▶ Merge sort has guaranteed $\Theta(n \log n)$ time and $\Theta(n)$ extra memory.

What to remember

- ▶ Selection and insertion sort are useful baselines, but their average and worst cases are quadratic.
- ▶ Merge sort splits by position and merges sorted halves.
- ▶ Merge sort has guaranteed $\Theta(n \log n)$ time and $\Theta(n)$ extra memory.
- ▶ Quicksort partitions around a pivot and is intended to be implemented in place.

What to remember

- ▶ Selection and insertion sort are useful baselines, but their average and worst cases are quadratic.
- ▶ Merge sort splits by position and merges sorted halves.
- ▶ Merge sort has guaranteed $\Theta(n \log n)$ time and $\Theta(n)$ extra memory.
- ▶ Quicksort partitions around a pivot and is intended to be implemented in place.
- ▶ Best and average quicksort are $\Theta(n \log n)$, but worst-case quicksort is $\Theta(n^2)$.

What to remember

- ▶ Selection and insertion sort are useful baselines, but their average and worst cases are quadratic.
- ▶ Merge sort splits by position and merges sorted halves.
- ▶ Merge sort has guaranteed $\Theta(n \log n)$ time and $\Theta(n)$ extra memory.
- ▶ Quicksort partitions around a pivot and is intended to be implemented in place.
- ▶ Best and average quicksort are $\Theta(n \log n)$, but worst-case quicksort is $\Theta(n^2)$.
- ▶ The quality of the pivot determines the shape of the quicksort recursion tree.

What to remember

- ▶ Selection and insertion sort are useful baselines, but their average and worst cases are quadratic.
- ▶ Merge sort splits by position and merges sorted halves.
- ▶ Merge sort has guaranteed $\Theta(n \log n)$ time and $\Theta(n)$ extra memory.
- ▶ Quicksort partitions around a pivot and is intended to be implemented in place.
- ▶ Best and average quicksort are $\Theta(n \log n)$, but worst-case quicksort is $\Theta(n^2)$.
- ▶ The quality of the pivot determines the shape of the quicksort recursion tree.

The central comparison: merge sort is reliable; quicksort is usually fast but pivot-sensitive.